

```
# Install required packages
```

```
!pip install -q opik datasets python-dotenv
```

```
!pip install -q transformers torch accelerate bitsandbytes  
sentencepiece protobuf
```

```
157.3/157.3 kB 6.4 MB/s eta
```

```
0:00:00
```

```
40.5/40.5 kB 2.0 MB/s eta
```

```
0:00:00
```

```
1.8/1.8 MB 36.5 MB/s eta
```

```
0:00:00
```

```
71.0/71.0 kB 4.8 MB/s eta
```

```
0:00:00
```

```
16.6/16.6 MB 65.5 MB/s eta
```

```
0:00:00
```

```
3.1/3.1 MB 67.4 MB/s eta
```

```
0:00:00
```

```
635.4/635.4 kB 38.6 MB/s eta
```

```
0:00:00
```

```
99.7/99.7 kB 6.5 MB/s eta
```

```
0:00:00
```

```
345.0/345.0 kB 22.5 MB/s eta
```

```
0:00:00
```

```
278.1/278.1 kB 17.4 MB/s eta
```

```
0:00:00
```

```
67.1/67.1 kB 4.7 MB/s eta
```

```
0:00:00
```

```
45.7/45.7 kB 4.5 MB/s eta
```

```
0:00:00
```

```
60.7/60.7 MB 13.5 MB/s eta
```

```
0:00:00
```

```
# Install rouge-score if not already installed
```

```
!pip install -q rouge-score
```

```
print("🟢 rouge-score package installed!")
```

```
Preparing metadata (setup.py) ...
```

```
# Install bert-score if not already installed
```

```
!pip install -q bert-score
```

```
print("🟢 bert-score package installed!")
```

```
0.0/61.1 kB ? eta -:--:-
```

```
61.1/61.1 kB 3.2 MB/s eta
```

```
0:00:00
```

```
# Import core libraries
```

```
import os
```

```
import json
```

```

import random
import time
import gc
from typing import Dict, List, Any, Optional
from dataclasses import dataclass

# Data and ML
import pandas as pd
from datasets import load_dataset
from tqdm.auto import tqdm

# Opik for evaluation
import opik
from opik import track, Opik
from opik.api_objects.opik_client import ApiError # Corrected import
path for ApiError
from opik.evaluation import evaluate
from opik.evaluation.metrics import base_metric, score_result

# LLM SDK
# NOTE: Groq exposes an OpenAI-compatible API, so we reuse the
# `openai` client
# everywhere by simply pointing it at Groq's base_url. No paid OpenAI
# key needed.
import openai

# Hugging Face Transformers
import torch
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    BitsAndBytesConfig,
    pipeline
)

print("✅ All packages imported successfully!")
print(f"🔥 PyTorch version: {torch.__version__}")
print(f"🔥 CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"🔥 GPU: {torch.cuda.get_device_name(0)}")

✅ All packages imported successfully!
✅ PyTorch version: 2.11.0+cu128
✅ CUDA available: True
✅ GPU: Tesla T4

```

Here's a single-line explanation for each package imported:

- **os:** Provides a way of using operating system dependent functionality.
- **json:** For working with JSON (JavaScript Object Notation) data.

- **random**: To generate pseudo-random numbers.
- **time**: For time-related functions.
- **gc**: Provides an interface to the garbage collector.
- **typing**: Supports type hints for better code readability and error checking.
- **dataclasses**: For creating simple data-holding classes.
- **pandas**: A powerful library for data manipulation and analysis.
- **datasets**: To easily load and prepare datasets for machine learning.
- **tqdm.auto**: For displaying smart progress bars.
- **opik**: (Assuming from context) A library for evaluation and tracking in machine learning.
- **openai**: The official Python client library for the OpenAI API.
- **anthropic**: The official Python client library for the Anthropic (Claude) API.
- **google.generativeai**: The official Python client library for Google's Generative AI models (Gemini/PaLM).
- **torch**: The PyTorch deep learning framework.
- **transformers**: Provides state-of-the-art pre-trained models for NLP tasks.
 - **AutoModelForCausalLM**: Automatically loads a causal language model.
 - **AutoTokenizer**: Automatically loads a tokenizer for a given model.
 - **BitsAndBytesConfig**: Configuration for 8-bit or 4-bit quantization to reduce memory usage.
 - **pipeline**: A high-level API to perform various NLP tasks with pre-trained models.

```
from huggingface_hub import login
login()
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:124: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
```

```
{"model_id": "3d6f7177625c445b854978a3b285089c", "version_major": 2, "version_minor": 0}
```

```
MODELS = {}
```

```
# OpenAI models removed as per user request.
```

```
MODELS["llama-3.1-8b-instant"] = {
    "provider": "groq",
    "model_name": "llama-3.1-8b-instant",
    "description": "Groq-hosted Llama 3.1 8B - fast, free tier",
    "enabled": True,
```

```

}
MODELS["llama-3.3-70b-versatile"] = {
    "provider": "groq",
    "model_name": "llama-3.3-70b-versatile",
    "description": "Groq-hosted Llama 3.3 70B - high quality, free tier",
    "enabled": True,
}
# Hugging Face Models - Local inference with .generate()
# These will be loaded locally and run on GPU if available

MODELS["llama-3.2-3b"] = {
    "provider": "huggingface",
    "model_name": "meta-llama/Llama-3.2-3B-Instruct",
    "description": "Meta's Llama 3.2 3B model (free, gated on Hugging Face)",
    "enabled": True,
    "load_in_4bit": True,
    "max_new_tokens": 100,
    "temperature": 0.01,
}

MODELS["gemma-2b-it"] = {
    "provider": "huggingface",
    "model_name": "google/gemma-4-E2B-it",
    "description": "Google's Gemma 4 2B Instruct model (free, open-weight)",
    "enabled": True,
    "load_in_4bit": False, # Explicitly set to False for CPU usage
    "max_new_tokens": 100,
    "temperature": 0.01,
}

# Disable HuggingFace models if no GPU is available
# The 'load_in_4bit' parameter is GPU-specific. Without a GPU, these models
# would load in full precision on CPU, which often leads to out-of-memory errors.
if not torch.cuda.is_available():
    print("⚠ No GPU detected. Disabling HuggingFace models configured with 'load_in_4bit' as it's GPU-specific and can cause out-of-memory errors on CPU.")
    # Create a list of keys to iterate over to allow dictionary modification
    for model_key in list(MODELS.keys()):
        config = MODELS[model_key]
        if config["provider"] == "huggingface" and config.get("load_in_4bit", False):
            MODELS[model_key]["enabled"] = False

```

```
        print(f"    - Disabled {model_key} (HuggingFace, 4-bit  
quantization requested)")
```

```
# Display configured models
```

```
print("☺ Available Models:\n")
```

```
enabled_models = {k: v for k, v in MODELS.items() if v.get("enabled",  
False)}
```

```
print(f"☐ {len(enabled_models)} models enabled\n")
```

```
☺ Available Models:
```

```
☐ 4 models enabled
```

```
for key, config in enabled_models.items():
```

```
    print(f"• {key}")
```

```
    print(f"    Provider: {config['provider']}")
```

```
    print(f"    Model: {config['model_name']}")
```

```
    print(f"    Description: {config['description']}")
```

```
    if config['provider'] == 'huggingface':
```

```
        print(f"        Quantization: {'4-bit' if config.get('load_in_4bit')"
```

```
else 'Full precision'}")
```

```
    print()
```

```
• llama-3.1-8b-instant
```

```
    Provider: groq
```

```
    Model: llama-3.1-8b-instant
```

```
    Description: Groq-hosted Llama 3.1 8B - fast, free tier
```

```
• llama-3.3-70b-versatile
```

```
    Provider: groq
```

```
    Model: llama-3.3-70b-versatile
```

```
    Description: Groq-hosted Llama 3.3 70B - high quality, free tier
```

```
• llama-3.2-3b
```

```
    Provider: huggingface
```

```
    Model: meta-llama/Llama-3.2-3B-Instruct
```

```
    Description: Meta's Llama 3.2 3B model (free, gated on Hugging Face)
```

```
    Quantization: 4-bit
```

```
• gemma-2b-it
```

```
    Provider: huggingface
```

```
    Model: google/gemma-4-E2B-it
```

```
    Description: Google's Gemma 4 2B Instruct model (free, open-weight)
```

```
    Quantization: Full precision
```

```
MODELS.items()
```

```
dict_items([('llama-3.1-8b-instant', {'provider': 'groq',
```

```
'model_name': 'llama-3.1-8b-instant', 'description': 'Groq-hosted
```

```
Llama 3.1 8B - fast, free tier', 'enabled': True}}, ('llama-3.3-70b-versatile', {'provider': 'groq', 'model_name': 'llama-3.3-70b-versatile', 'description': 'Groq-hosted Llama 3.3 70B - high quality, free tier', 'enabled': True}}, ('llama-3.2-3b', {'provider': 'huggingface', 'model_name': 'meta-llama/Llama-3.2-3B-Instruct', 'description': "Meta's Llama 3.2 3B model (free, gated on Hugging Face)", 'enabled': True, 'load_in_4bit': True, 'max_new_tokens': 100, 'temperature': 0.01}), ('gemma-2b-it', {'provider': 'huggingface', 'model_name': 'google/gemma-4-E2B-it', 'description': "Google's Gemma 4 2B Instruct model (free, open-weight)", 'enabled': True, 'load_in_4bit': False, 'max_new_tokens': 100, 'temperature': 0.01}))])
```

```
class HuggingFaceModelManager:
```

```
    """
```

```
    Manager for loading and running Hugging Face models locally.
    Handles model caching, quantization, and cleanup.
    """
```

```
    def __init__(self):
```

```
        self.loaded_models = {}
```

```
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
        print(f"📦 HuggingFace inference will use: {self.device}")
```

```
    def load_model(self, model_name: str, load_in_4bit: bool = True):
```

```
        """
```

```
        Load a Hugging Face model with optional 4-bit quantization.
```

```
        Args:
```

```
            model_name: HuggingFace model identifier
```

```
            load_in_4bit: Whether to use 4-bit quantization
```

```
        """
```

```
        if model_name in self.loaded_models:
```

```
            print(f"🔄 Using cached model: {model_name}")
```

```
            return self.loaded_models[model_name]
```

```
        print(f"📦 Loading model: {model_name}")
```

```
        print(f"🔢 Quantization: {'4-bit' if load_in_4bit else 'None'}")
```

```
        try:
```

```
            # Load tokenizer with token
```

```
            tokenizer = AutoTokenizer.from_pretrained(
```

```
                model_name,
```

```
                trust_remote_code=True,
```

```
            )
```

```
            # Set padding token if not exists
```

```
            if tokenizer.pad_token is None:
```

```

        tokenizer.pad_token = tokenizer.eos_token

    # Configure quantization
    if load_in_4bit and torch.cuda.is_available():
        quantization_config = BitsAndBytesConfig(
            load_in_4bit=True,
            bnb_4bit_compute_dtype=torch.float16,
            bnb_4bit_use_double_quant=True,
            bnb_4bit_quant_type="nf4"
        )

        model = AutoModelForCausalLM.from_pretrained(
            model_name,
            quantization_config=quantization_config,
            device_map="auto",
            trust_remote_code=True,
            torch_dtype=torch.float16,
        )
    else:
        # CPU or full precision
        model = AutoModelForCausalLM.from_pretrained(
            model_name,
            trust_remote_code=True,
            torch_dtype=torch.float16 if
torch.cuda.is_available() else torch.float32,
        )
        if not load_in_4bit: # Only move to device if not
using device_map
            model = model.to(self.device)

    model.eval() # Set to evaluation mode

    self.loaded_models[model_name] = {
        "model": model,
        "tokenizer": tokenizer
    }

    print(f"    □ Model loaded successfully")

    # Show memory usage if on GPU
    if torch.cuda.is_available():
        memory_allocated = torch.cuda.memory_allocated() /
1024**3
        print(f"    □ GPU Memory: {memory_allocated:.2f} GB")

    return self.loaded_models[model_name]

except Exception as e:
    error_msg = str(e)

```

```

        if "gated" in error_msg.lower() or "access" in
error_msg.lower():
            print(f"    ❌ Access denied: This is a gated model")
            print(f"    Steps to fix:")
            print(f"        1. Go to
https://huggingface.co/{model_name}")
            print(f"        2. Accept the model's license
agreement")
            print(f"        3. Create a token at
https://huggingface.co/settings/tokens")
            print(f"        4. Add it to Colab Secrets as
'HUGGINGFACE_API_KEY'")
        else:
            print(f"    ❌ Error loading model: {error_msg}")
            raise

def generate(
    self,
    model_name: str,
    prompt: str,
    max_new_tokens: int = 100,
    temperature: float = 0.01,
    load_in_4bit: bool = True
) -> str:
    """
    Generate text using a Hugging Face model.

    Args:
        model_name: HuggingFace model identifier
        prompt: Input prompt
        max_new_tokens: Maximum tokens to generate
        temperature: Sampling temperature
        load_in_4bit: Whether to use 4-bit quantization

    Returns:
        Generated text
    """
    # Load model if not cached
    model_dict = self.load_model(model_name, load_in_4bit)
    model = model_dict["model"]
    tokenizer = model_dict["tokenizer"]

    # Tokenize input
    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=512
    )

```



```

# Move to device
#if not load_in_4bit or not torch.cuda.is_available():
inputs = {k: v.to(self.device) for k, v in inputs.items()}

# Generate
with torch.no_grad():
    outputs = model.generate(
        **inputs,
        max_new_tokens=max_new_tokens,
        temperature=temperature,
        do_sample=temperature > 0,
        pad_token_id=tokenizer.pad_token_id,
        eos_token_id=tokenizer.eos_token_id,
    )

# Decode output
generated_text = tokenizer.decode(
    outputs[0][inputs['input_ids'].shape[1]:], # Only new
tokens
    skip_special_tokens=True
)

return generated_text.strip()

def clear_cache(self):
    """Clear all loaded models from memory."""
    print("❏ Clearing model cache...")
    for model_name in list(self.loaded_models.keys()):
        del self.loaded_models[model_name]
    self.loaded_models = {}

# Force garbage collection
gc.collect()
if torch.cuda.is_available():
    torch.cuda.empty_cache()
    print(f"❏ Cache cleared. GPU Memory:
{torch.cuda.memory_allocated() / 1024**3:.2f} GB")
else:
    print("❏ Cache cleared")

# Initialize the model manager
hf_manager = HuggingFaceModelManager()
print("❏ Hugging Face Model Manager initialized!")

❏ HuggingFace inference will use: cuda
❏ Hugging Face Model Manager initialized!

```

custom **HuggingFaceModelManager** here is an advanced pattern specifically for managing a suite of models with custom configurations and memory optimizations for this comparison experiment

In summary: If your goal was just to use one Hugging Face model for a straightforward task, you absolutely would use pipeline or direct AutoModel/AutoTokenizer loading

□ Dataset Loading

We'll use the **SST-2** (Stanford Sentiment Treebank) dataset - a widely-used benchmark for binary sentiment classification.

Dataset Details

- **Task:** Binary sentiment classification (positive/negative)
- **Domain:** Movie reviews
- **Size:** 67K training examples, 872 validation examples
- **Source:** Extracted from Rotten Tomatoes reviews

```
# Reinstall datasets and huggingface_hub to resolve potential version
conflicts or corrupted installations
print("Reinstalling 'datasets' and 'huggingface_hub'...")
!pip uninstall -y datasets huggingface_hub
!pip install -q datasets huggingface_hub
print("Reinstallation complete. Please rerun the dataset loading cell
above.")
```

```
Reinstalling 'datasets' and 'huggingface_hub'...
```

```
Found existing installation: datasets 4.0.0
```

```
Uninstalling datasets-4.0.0:
```

```
  Successfully uninstalled datasets-4.0.0
```

```
Found existing installation: huggingface_hub 1.19.0
```

```
Uninstalling huggingface_hub-1.19.0:
```

```
  Successfully uninstalled huggingface_hub-1.19.0
```

```
----- 555.1/555.1 kB 13.9 MB/s eta
0:00:00
```

```
----- 721.1/721.1 kB 29.5 MB/s eta
0:00:00
```

```
----- 48.9/48.9 MB 23.1 MB/s eta
0:00:00
```

```
plete. Please rerun the dataset loading cell above.
```

```
# Load SST-2 dataset from Hugging Face
```

```
print("Loading SST-2 dataset...")
```

```
dataset = load_dataset("nyu-ml/glu", "sst2")
```

```
# We'll use a subset of the validation set for faster evaluation
eval_dataset = dataset["validation"]
```

```
# Sample 100 examples for our evaluation (adjust based on your needs)
sample_size = 100
indices = random.sample(range(len(eval_dataset)), sample_size)
eval_subset = eval_dataset.select(indices)
```

```
print(f"📦 Loaded {len(eval_subset)} examples for evaluation")
print(f"\n📦 Label distribution:")
print(f"   Negative (0): {sum(1 for x in eval_subset if x['label'] == 0)} examples")
print(f"   Positive (1): {sum(1 for x in eval_subset if x['label'] == 1)} examples")
```

Loading SST-2 dataset...

```
{"model_id": "la478e064f26487aaaaf78fa59be4e61", "version_major": 2, "version_minor": 0}
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```
{"model_id": "2fe56764b9a54962a65b6c3f748fe050", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "f2212f2c1b9849a8b0c1a1619ca21dd2", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "cbf66ff5ae854c828b024bf01d4de2a6", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1761c01774534ef6873703f483d7324c", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "d344266c3b954ce8b848d21ff7dd56e3", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "c3868184d6b449109937a7349689b694", "version_major": 2, "version_minor": 0}
```

📦 Loaded 100 examples for evaluation

📦 Label distribution:
 Negative (0): 49 examples
 Positive (1): 51 examples

```

print("\n Sample Examples:\n")
for i in range(3):
    example = eval_subset[i]
    sentiment = "Positive 😊" if example['label'] == 1 else
    "Negative 😞"
    print(f"Example {i+1}:")
    print(f"  Text: {example['sentence']}")
    print(f"  Label: {sentiment}")
    print()

```

□ Sample Examples:

Example 1:

Text: if there 's one thing this world needs less of , it 's movies about college that are written and directed by people who could n't pass an entrance exam .

Label: Negative 😞

Example 2:

Text: the film tries too hard to be funny and tries too hard to be hip .

Label: Negative 😞

Example 3:

Text: offers much to enjoy ... and a lot to mull over in terms of love , loyalty and the nature of staying friends .

Label: Positive 😊

□ Prompt Design

Consistent prompting across models is crucial for fair comparison. We'll use a simple, clear prompt that works well with all models.

Prompt Strategy

1. **Clear instruction:** Tell the model exactly what to do
2. **Format specification:** Request structured output (JSON)
3. **Label options:** Explicitly state the two classes
4. **No bias:** Avoid suggesting an answer

```

SENTIMENT_PROMPT = """You are a sentiment analysis expert. Analyze the
sentiment of the following text and classify it as either "positive"
or "negative".

```

Text: {text}

Respond ONLY with a JSON object in this exact format:
{"sentiment": "positive"} or {"sentiment": "negative"}

Response: ""

Test the prompt with an example

sample_text = "This movie was absolutely fantastic!"

formatted_prompt = SENTIMENT_PROMPT.format(text=sample_text)

print(" Prompt Template:\n")

print(formatted_prompt)

Prompt Template:

You are a sentiment analysis expert. Analyze the sentiment of the following text and classify it as either "positive" or "negative".

Text: This movie was absolutely fantastic!

Respond ONLY with a JSON object in this exact format:
{"sentiment": "positive"} or {"sentiment": "negative"}

Response:

eval_subset

```
Dataset({
  features: ['sentence', 'label', 'idx'],
  num_rows: 100
})
```

Convert HF dataset to Opik format

```
def create_opik_dataset(hf_dataset, dataset_name: str):
    """
```

Convert Hugging Face dataset to Opik dataset format.

Args:

hf_dataset: Hugging Face dataset

dataset_name: Name for the Opik dataset

Returns:

Opik dataset object

"""

Check if dataset already exists

try:

existing_dataset = opik_client.get_dataset(dataset_name)

print(f" Using existing dataset: {dataset_name}")

```

        return existing_dataset
    except:
        print(f"❌ Creating new dataset: {dataset_name}")

    # Prepare dataset items
    dataset_items = []
    for item in hf_dataset:
        dataset_items.append({
            "input": {
                "text": item["sentence"]
            },
            "expected_output": {
                "label": item["label"],
                "sentiment": "positive" if item["label"] == 1 else
"negative"
            }
        })

    # Create Opik dataset
    dataset = opik_client.create_dataset(
        name=dataset_name,
        description="SST-2 sentiment analysis evaluation dataset"
    )

    # Insert items
    dataset.insert(dataset_items)

    print(f"❌ Created dataset with {len(dataset_items)} items")
    return dataset

import os
from getpass import getpass

import opik
from opik import Opik

# 1. Double check that your settings match your dashboard workspace
string exactly
os.environ["OPIK_API_KEY"] = "3LDp2VzfLVzrSCIWaBBqY15KU ".strip() #
Added .strip() to remove potential whitespace
os.environ["OPIK_WORKSPACE"] = "rahil_eval_1"
os.environ["GROQ_API_KEY"] = getpass("Enter GROQ_API_KEY: ")
# 2. FORCE the SDK to overwrite its internal cache with the new
variables
opik.configure(force=True)

# Groq's OpenAI-compatible endpoint. The `openai` client + this
base_url are used
# for all hosted-model calls in this notebook. LiteLLM (used by Opik's
LLM-judge

```

```
# metrics) reads GROQ_API_KEY from the environment automatically.
GROQ_BASE_URL = "https://api.groq.com/openai/v1"
```

```
# 3. Re-instantiate a completely blank client instance
opik_client = Opik()
```

```
# 4. Re-run your conversion function
opik_dataset = create_opik_dataset(eval_subset,
"sst2_sentiment_eval_v1")
```

```
Enter GROQ_API_KEY: .....
```

```
OPIK: Your Opik API key is available in your account settings, can be
found at https://www.comet.com/api/my/settings/ for Opik cloud
```

```
Please enter your Opik API key:.....
```

```
OPIK: Configuration saved to file: /root/.opik.config
OPIK: Configuration completed successfully. Traces will be logged to
'Opik Demo Agent Observability' project. To change the destination
project, see:
https://www.comet.com/docs/opik/tracing/log_traces#configuring-the-
project-name
```

```
□ Using existing dataset: sst2_sentiment_eval_v1
```

```
opik_dataset
```

```
<opik.api_objects.dataset.dataset.Dataset at 0x7f7c665c6300>
```

□ Model Inference Functions

We'll create helper functions to:

1. Call each LLM with the sentiment prompt
2. Parse the response
3. Handle errors gracefully

```
def parse_sentiment_response(response_text: str) -> str:
    """
    Parse the model's response to extract sentiment.
    Handles various response formats.
    """
    if not response_text:
        return "negative"

    response_text = response_text.strip().lower()
```

```

# Try to parse as JSON
try:
    if "{" in response_text and "}" in response_text:
        start = response_text.find("{")
        end = response_text.rfind("}") + 1
        json_str = response_text[start:end]
        data = json.loads(json_str)

        if "sentiment" in data:
            sentiment = data["sentiment"].lower()
            if sentiment in ["positive", "negative"]:
                return sentiment
except:
    pass

# Fallback: keyword matching
if "positive" in response_text:
    return "positive"
elif "negative" in response_text:
    return "negative"

return "negative"

#
=====
=====
# GROQ INFERENCE (free tier, OpenAI-compatible API)
#
=====
=====

def get_groq_client():
    """Return an OpenAI client pointed at Groq's OpenAI-compatible
    endpoint."""
    return openai.OpenAI(
        api_key=os.environ["GROQ_API_KEY"],
        base_url=GROQ_BASE_URL,
    )

def call_groq(text: str, model_name: str) -> Dict[str, Any]:
    """Call Groq API for sentiment analysis."""
    try:
        client = get_groq_client()

        prompt = SENTIMENT_PROMPT.format(text=text)

        response = client.chat.completions.create(

```



```

        model=model_name,
        messages=[
            {"role": "user", "content": prompt}
        ],
        max_tokens=100,
        temperature=0.0,
    )

    response_text = response.choices[0].message.content
    predicted_sentiment = parse_sentiment_response(response_text)

    return {
        "raw_response": response_text,
        "predicted_sentiment": predicted_sentiment,
        "success": True,
        "error": None,
        "tokens": response.usage.total_tokens if hasattr(response,
'usage') else None,
    }

except Exception as e:
    return {
        "raw_response": None,
        "predicted_sentiment": "negative",
        "success": False,
        "error": f"Groq API error: {str(e)}",
        "tokens": None,
    }

def call_huggingface(text: str, model_config: Dict) -> Dict[str, Any]:
    """
    Call Hugging Face model using local inference with .generate().

    Args:
        text: Input text to analyze
        model_config: Model configuration dict

    Returns:
        Dict with prediction and metadata
    """
    try:
        model_name = model_config["model_name"]
        max_new_tokens = model_config.get("max_new_tokens", 100)
        temperature = model_config.get("temperature", 0.01)
        load_in_4bit = model_config.get("load_in_4bit", True)

        # Format prompt
        prompt = SENTIMENT_PROMPT.format(text=text)

        # Add chat template formatting for instruct models

```

```

    # Most instruct models expect a specific format
    tokenizer_check = AutoTokenizer.from_pretrained(model_name,

#trust_remote_code=True

)

    if hasattr(tokenizer_check, 'apply_chat_template'):
        # Use chat template if available
        messages = [{"role": "user", "content": prompt}]
        try:
            formatted_prompt =
tokenizer_check.apply_chat_template(
                messages,
                tokenize=False,
                add_generation_prompt=True
            )
        except:
            # Fallback if chat template fails
            formatted_prompt = prompt
    else:
        formatted_prompt = prompt

    # Generate response
    response_text = hf_manager.generate(
        model_name=model_name,
        prompt=formatted_prompt,
        max_new_tokens=max_new_tokens,
        temperature=temperature,
        load_in_4bit=load_in_4bit
    )

    # Parse sentiment
    predicted_sentiment = parse_sentiment_response(response_text)

    return {
        "raw_response": response_text,
        "predicted_sentiment": predicted_sentiment,
        "success": True,
        "error": None,
        "tokens": None, # Could calculate with tokenizer if
needed
    }

except Exception as e:
    return {
        "raw_response": None,
        "predicted_sentiment": "negative",
        "success": False,
        "error": f"HuggingFace generation error: {str(e)}",
        "tokens": None,

```

```

    }

def call_llm(text: str, model_config: Dict) -> Dict[str, Any]:
    """
    Unified function to call any LLM based on provider.

    Args:
        text: Input text to analyze
        model_config: Model configuration dict

    Returns:
        Dict with prediction and metadata
    """
    provider = model_config["provider"]
    model_name = model_config["model_name"]

    # Removed OpenAI provider branch as per user request.

    if provider == "groq":
        return call_groq(text, model_name)
    elif provider == "huggingface":
        return call_huggingface(text, model_config) # Pass full
config for HF
    else:
        return {
            "raw_response": None,
            "predicted_sentiment": "negative",
            "success": False,
            "error": f"Unknown provider: {provider}",
            "tokens": None,
        }

print("□ All inference functions defined (with local HF generation)!")
□ All inference functions defined (with local HF generation)!

# Test all configured models
test_text = "This movie was absolutely fantastic!"

print("□ Testing Inference Functions\n")
print("=" * 80)
print(f"Test input: '{test_text}'\n")

for model_key, model_config in MODELS.items():
    if not model_config.get("enabled", False):
        continue

    print(f"\n{'='*80}")
    print(f"□ Testing: {model_key}")

```

```

print(f"    Provider: {model_config['provider']}")
print(f"    Model: {model_config['model_name']}")
print("-" * 80)

try:
    result = call_llm(test_text, model_config)

    if result["success"]:
        print(f"    Success")
        print(f"    Predicted:
{result['predicted_sentiment'].upper()}")
        print(f"    Raw response:")
        print(f"    {result['raw_response'][:200]}...")
        if result.get('tokens'):
            print(f"    Tokens used: {result['tokens']}")
    else:
        print(f"    Failed")
        print(f"    Error: {result['error']}")

except Exception as e:
    print(f"    Exception: {str(e)}")

print()

print("=" * 80)
print("All tests complete!")

```

Testing Inference Functions

```

=====
=====
Test input: 'This movie was absolutely fantastic!'

=====
=====
Testing: llama-3.1-8b-instant
Provider: groq
Model: llama-3.1-8b-instant
-----
-----
    Success
    Predicted: POSITIVE
    Raw response:
{"sentiment": "positive"}...
    Tokens used: 106

=====
=====

```

```
□ Testing: llama-3.3-70b-versatile
  Provider: groq
  Model: llama-3.3-70b-versatile
```

```
-----
□ Success
Predicted: POSITIVE
Raw response:
{"sentiment": "positive"}...
Tokens used: 106
```

```
=====
□ Testing: llama-3.2-3b
  Provider: huggingface
  Model: meta-llama/Llama-3.2-3B-Instruct
```

```
-----
□ Failed
Error: HuggingFace generation error: You are trying to access a
gated repo.
Make sure to have access to it at
https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct.
401 Client Error. (Request ID: Root=1-6a422ce1-
5913dcc135495660692c2b2a;c40c8d9f-c4f0-4387-a51f-f81a2a6ea9f7)
```

```
Cannot access gated repo for url
https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct/resolve/main/
config.json.
Access to model meta-llama/Llama-3.2-3B-Instruct is restricted. You
must have access to it and be authenticated to access it. Please log
in.
```

```
=====
□ Testing: gemma-2b-it
  Provider: huggingface
  Model: google/gemma-4-E2B-it
```

```
-----
{"model_id": "a14e1d9204fa49ec90bdf5b6f762c32f", "version_major": 2, "vers
ion_minor": 0}

{"model_id": "7c0b373834db4409a2c221cdf7fef9c", "version_major": 2, "vers
ion_minor": 0}
```

```
{"model_id": "19b42a8beb564883abfb13103bd73bb8", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "c3cb83e58fd74686a630b9253ed8ee93", "version_major": 2, "version_minor": 0}
```

```
❑ Loading model: google/gemma-4-E2B-it  
❑ Quantization: None
```

```
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!
```

```
{"model_id": "072ec4e770ab41fcb37d6dc1e05624ee", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1fc35d4c553a4a4aa1f93f32d812c45a", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "3e70d70bec1d4cb3a11409d3e1ab6508", "version_major": 2, "version_minor": 0}
```

```
❑ Model loaded successfully  
❑ GPU Memory: 9.52 GB  
❑ Success  
Predicted: POSITIVE  
Raw response:  
{"sentiment": "positive"}...
```

```
=====
```

```
❑ All tests complete!
```

```
MODELS.keys()
```

```
result = call_llm(test_text, MODELS["llama-3.3-70b-versatile"])  
result
```

```
{'raw_response': '{"sentiment": "positive"}',  
 'predicted_sentiment': 'positive',  
 'success': True,  
 'error': None,  
 'tokens': 106}
```

```
result = call_llm(test_text, MODELS["gemma-2b-it"])  
result
```

```
✳ Using cached model: google/gemma-4-E2B-it
```

```
{'raw_response': '{"sentiment": "positive"}',  
 'predicted_sentiment': 'positive',  
 'success': True,  
 'error': None,  
 'tokens': None}
```

```

from opik.evaluation.metrics import base_metric, score_result

class AccuracyMetric(base_metric.BaseMetric):
    """Accuracy metric for binary sentiment classification."""

    def __init__(self, name: str = "accuracy"):
        self.name = name

    def score(
        self,
        predicted_sentiment: str,
        expected_sentiment: str,
        **kwargs
    ) -> score_result.ScoreResult:
        """Score a single prediction."""
        is_correct = (predicted_sentiment.lower() ==
expected_sentiment.lower())

        return score_result.ScoreResult(
            name=self.name,
            value=1.0 if is_correct else 0.0,
            reason=f"Predicted: {predicted_sentiment}, Expected:
{expected_sentiment}"
        )

class F1Metric(base_metric.BaseMetric):
    """F1 score approximation (treats as accuracy for individual
samples)."""

    def __init__(self, name: str = "f1_score"):
        self.name = name

    def score(
        self,
        predicted_sentiment: str,
        expected_sentiment: str,
        **kwargs
    ) -> score_result.ScoreResult:
        """Score a single prediction - returns 1.0 if correct, 0.0 if
incorrect."""
        is_correct = (predicted_sentiment.lower() ==
expected_sentiment.lower())

        return score_result.ScoreResult(
            name=self.name,
            value=1.0 if is_correct else 0.0,
            reason="Correct prediction" if is_correct else "Incorrect
prediction"

```

```

    )

class PrecisionMetric(base_metric.BaseMetric):
    """Precision metric for positive class (no None values)."""

    def __init__(self, name: str = "precision"):
        self.name = name

    def score(
        self,
        predicted_sentiment: str,
        expected_sentiment: str,
        **kwargs
    ) -> score_result.ScoreResult:
        """
        Score for precision calculation.
        Returns 1.0 for TP, 0.0 for FP, and 0.0 for TN/FN (not
        applicable).
        """
        pred = predicted_sentiment.lower()
        true = expected_sentiment.lower()

        if pred == "positive":
            # Predicted positive: check if correct
            is_correct = (true == "positive")
            return score_result.ScoreResult(
                name=self.name,
                value=1.0 if is_correct else 0.0,
                reason="True Positive" if is_correct else "False
Positive"
            )
        else:
            # Predicted negative: return 0.0 (not counted for
            precision)
            # Using 0.0 instead of None to avoid aggregation errors
            return score_result.ScoreResult(
                name=self.name,
                value=0.0,
                reason="Predicted negative (not applicable for
precision)"
            )

class RecallMetric(base_metric.BaseMetric):
    """Recall metric for positive class (no None values)."""

    def __init__(self, name: str = "recall"):
        self.name = name

```



```

def score(
    self,
    predicted_sentiment: str,
    expected_sentiment: str,
    **kwargs
) -> score_result.ScoreResult:
    """
    Score for recall calculation.
    Returns 1.0 for TP, 0.0 for FN, and 0.0 for TN/FP (not
    applicable).
    """
    pred = predicted_sentiment.lower()
    true = expected_sentiment.lower()

    if true == "positive":
        # Actually positive: check if we caught it
        is_correct = (pred == "positive")
        return score_result.ScoreResult(
            name=self.name,
            value=1.0 if is_correct else 0.0,
            reason="True Positive" if is_correct else "False
Negative"
        )
    else:
        # Actually negative: return 0.0 (not counted for recall)
        # Using 0.0 instead of None to avoid aggregation errors
        return score_result.ScoreResult(
            name=self.name,
            value=0.0,
            reason="True negative (not applicable for recall)"
        )

print(" Custom classification metrics created!")
print(" • AccuracyMetric: Overall correctness")
print(" • F1Metric: F1 score approximation")
print(" • PrecisionMetric: Precision for positive class")
print(" • RecallMetric: Recall for positive class")

 Custom classification metrics created!
  • AccuracyMetric: Overall correctness
  • F1Metric: F1 score approximation
  • PrecisionMetric: Precision for positive class
  • RecallMetric: Recall for positive class

def create_evaluation_task(model_name: str, model_config: Dict):
    """
    Create an evaluation task function for a specific model.

    Args:

```

```

    model_name: Name/identifier for the model
    model_config: Model configuration dictionary

Returns:
    Task function compatible with Opik's evaluate()
"""
@track(project_name=f"sentiment_eval_{model_name}")
def evaluation_task(item: Dict) -> Dict:
    """
    Task function that Opik calls for each dataset item.

    Args:
        item: Dataset item with 'input' and 'expected_output'

    Returns:
        Dict with input, output, and expected output for metrics
    """
    # Extract input text
    text = item["input"]["text"]
    expected_sentiment = item["expected_output"]["sentiment"]

    # Run inference
    result = call_llm(text, model_config)

    # Return in Opik format
    return {
        "input": text,
        "output": result["predicted_sentiment"],
        "predicted_sentiment": result["predicted_sentiment"],
        "expected_sentiment": expected_sentiment,
        "raw_response": result["raw_response"],
        "model_name": model_name,
    }

    return evaluation_task

print("✅ Evaluation task creator defined!")
✅ Evaluation task creator defined!

from opik.evaluation.metrics import Equals
# Initialize metrics
scoring_metrics = [
    AccuracyMetric(),
    F1Metric(),
    PrecisionMetric(),
    RecallMetric(),
]

# Store experiment results

```

```

experiment_results = {}

print("\n Starting evaluation for all models...\n")
print("=" * 70)

for model_key, model_config in MODELS.items():
    print(f"\n Evaluating {model_key}...")
    print(f"    Model: {model_config['model_name']}")
    print(f"    Provider: {model_config['provider']}")
    print("-" * 70)

    try:
        # Create evaluation task for this model
        task = create_evaluation_task(model_key, model_config)

        # Run evaluation
        experiment = evaluate(
            dataset=opik_dataset,
            task=task,
            scoring_metrics=scoring_metrics,
            experiment_name=f"sentiment_{model_key}",
        )

        # Store results
        experiment_results[model_key] = experiment

        print(f"\n Completed evaluation for {model_key}")
        print(f"    Experiment URL: {experiment.experiment_url if
hasattr(experiment, 'experiment_url') else 'Available in Opik
dashboard'}")

    except Exception as e:
        print(f"\n Error evaluating {model_key}: {str(e)}")
        continue

print("\n" + "=" * 70)
print("\n All evaluations completed!")
print("\n View detailed results in your Opik dashboard:")
print(f"    https://www.comet.com/{os.environ['OPIK_WORKSPACE']}/opik")

```

Starting evaluation for all models...

```

=====

 Evaluating llama-3.1-8b-instant...
    Model: llama-3.1-8b-instant
    Provider: groq
-----

```

```

{"model_id": "d25095ee37b64222acf3b6152a823cbc", "version_major": 2, "version_minor": 0}

```

OPIK: Started logging traces to the "Opik Demo Agent Observability" project at
https://www.comet.com/opik/api/v1/session/redirect/projects/?trace_id=019f12a4-1c96-7855-afa2-384e9c64a4c7&path=aHR0cHM6Ly93d3cuY29tZXQuY29tL29waWsvYXBpLw==.

— sst2_sentiment_eval_v1 (100 samples) —

Total time: 00:00:22
Number of samples: 100

accuracy: 0.6100 (avg)
f1_score: 0.6100 (avg)
precision: 0.1300 (avg)
recall: 0.1300 (avg)

Uploading results to Opik ...

View the results

]8;id=984289;https://www.comet.com/opik/api/v1/session/redirect/experiments/?experiment_id=019f12a4-1618-7cb1-b935-977cb3c4d892&dataset_id=019f0daf-b233-7065-8815-091e59305505&path=aHR0cHM6Ly93d3cuY29tZXQuY29tL29waWsvYXBpLw==\in your Opik dashboard]8;;\.

☐ Completed evaluation for llama-3.1-8b-instant

Experiment URL:

https://www.comet.com/opik/api/v1/session/redirect/experiments/?experiment_id=019f12a4-1618-7cb1-b935-977cb3c4d892&dataset_id=019f0daf-b233-7065-8815-091e59305505&path=aHR0cHM6Ly93d3cuY29tZXQuY29tL29waWsvYXBpLw==

☐ Evaluating llama-3.3-70b-versatile...

Model: llama-3.3-70b-versatile

Provider: groq

{
 "model_id": "aaa1915d2018482cbf0eae7404377431",
 "version_major": 2,
 "version_minor": 0
}

— sst2_sentiment_eval_v1 (100 samples) —

Total time: 00:00:23
Number of samples: 100

accuracy: 0.6400 (avg)
f1_score: 0.6400 (avg)
precision: 0.1500 (avg)
recall: 0.1500 (avg)

Uploading results to Opik ...

View the results

]8;id=403368;https://www.comet.com/opik/api/v1/session/redirect/experiments/?experiment_id=019f12a4-7bf5-7618-93a6-cfb2cb3e69c8&dataset_id=019f0daf-b233-7065-8815-091e59305505&path=aHR0cHM6Ly93d3cuY29tZXQuY29tL29waWsvYXBpLw==\in your Opik dashboard]8;;\.

☐ Completed evaluation for llama-3.3-70b-versatile

Experiment URL:

https://www.comet.com/opik/api/v1/session/redirect/experiments/?experiment_id=019f12a4-7bf5-7618-93a6-cfb2cb3e69c8&dataset_id=019f0daf-b233-7065-8815-091e59305505&path=aHR0cHM6Ly93d3cuY29tZXQuY29tL29waWsvYXBpLw==

☐ Evaluating llama-3.2-3b...

Model: meta-llama/Llama-3.2-3B-Instruct

Provider: huggingface

{
 "model_id": "5826c142d13e4bd8aa250a8e6d227af9",
 "version_major": 2,
 "version_minor": 0
}

┌ sst2_sentiment_eval_v1 (100 samples) ─┐

Total time: 00:00:01

Number of samples: 100

accuracy: 0.5000 (avg)

f1_score: 0.5000 (avg)

precision: 0.0000 (avg)

recall: 0.0000 (avg)

Uploading results to Opik ...

View the results

]8;id=652061;https://www.comet.com/opik/api/v1/session/redirect/experiments/?experiment_id=019f12a4-e258-7601-a6b7-ecf14faa6b6a&dataset_id=019f0daf-b233-7065-8815-091e59305505&path=aHR0cHM6Ly93d3cuY29tZXQuY29tL29waWsvYXBpLw==\in your Opik dashboard]8;;\.

☐ Completed evaluation for llama-3.2-3b

Experiment URL:

https://www.comet.com/opik/api/v1/session/redirect/experiments/?

```
experiment_id=019f12a4-e258-7601-a6b7-ecf14faa6b6a&dataset_id=019f0daf-b233-7065-8815-091e59305505&path=aHR0cHM6Ly93d3cuY29tZXQuY29tL29waWsvYXBpLw==
```

```

❑ Evaluating gemma-2b-it...
  Model: google/gemma-4-E2B-it
  Provider: huggingface

```

```
{"model_id": "63e79f9b188a49a19c2c82649b055cbb", "version_major": 2, "version_minor": 0}
```

[illegible]

[illegible]

```
✧ Using cached model: google/gemma-4-E2B-it
✧ Using cached model: google/gemma-4-E2B-it
✧ Using cached model: google/gemma-4-E2B-it
✧ Using cached model: google/gemma-4-E2B-it
✧ Using cached model: google/gemma-4-E2B-it
✧ Using cached model: google/gemma-4-E2B-it
✧ Using cached model: google/gemma-4-E2B-it
✧ Using cached model: google/gemma-4-E2B-it
✧ Using cached model: google/gemma-4-E2B-it
✧ Using cached model: google/gemma-4-E2B-it
✧ Using cached model: google/gemma-4-E2B-it
✧ Using cached model: google/gemma-4-E2B-it
```

```
┌ sst2_sentiment_eval_v1 (100 samples) ─┐
```

```
Total time:      00:10:36
Number of samples: 100
```

```
accuracy: 0.8600 (avg)
f1_score: 0.8600 (avg)
precision: 0.3700 (avg)
recall: 0.3700 (avg)
```

Uploading results to Opik ...

View the results

```
]8;id=951973;https://www.comet.com/opik/api/v1/session/redirect/experiments/?experiment_id=019f12a4-f2c8-7510-8760-648a7c560c81&dataset_id=019f0daf-b233-7065-8815-091e59305505&path=aHR0cHM6Ly93d3cuY29tZXQuY29tL29waWsvYXBpLw==\in your Opik dashboard]8;;\.
```

☐ Completed evaluation for gemma-2b-it

Experiment URL:

```
https://www.comet.com/opik/api/v1/session/redirect/experiments/?experiment_id=019f12a4-f2c8-7510-8760-648a7c560c81&dataset_id=019f0daf-b233-7065-8815-091e59305505&path=aHR0cHM6Ly93d3cuY29tZXQuY29tL29waWsvYXBpLw==
```

```
=====
☐ All evaluations completed!
```

☐ View detailed results in your Opik dashboard:

```
https://www.comet.com/rahil_eval_1/opik
```


Why Custom Metrics for Opik Integration?

It's a valid question why custom metrics were implemented instead of using off-the-shelf metrics from libraries like scikit-learn. The primary reasons are deeply tied to how the Opik evaluation framework functions and the specific requirements of this evaluation setup:

1. **Opik Framework Compatibility:** The `opik.evaluation.evaluate` function is designed to work with metrics that are subclasses of `opik.evaluation.metrics.base_metric.BaseMetric`. Our custom `AccuracyMetric`, `F1Metric`, `PrecisionMetric`, and `RecallMetric` inherit from this base class, making them compatible with Opik's evaluation pipeline.
2. **Per-Sample Scoring:** Opik performs evaluation on a per-sample basis. For each item in the dataset, it calls the `score` method of every configured metric. Standard library metrics typically calculate a single, aggregated score (e.g., one F1 score for the entire dataset). Our custom metrics are designed to return a `1.0` or `0.0` for *each individual sample*, indicating correctness or specific conditions (True Positive, False Positive, etc.). Opik then aggregates these individual scores to compute the overall metric value for the experiment.
3. **Structured ScoreResult Output:** The `score` method within an Opik `BaseMetric` expects to return a `score_result.ScoreResult` object. This object provides a standardized way to package the score's value, name, and a descriptive reason for that score. This structure is crucial for Opik to properly record and display detailed evaluation results in the dashboard.
4. **Tailored Logic for Aggregation:** For metrics like Precision and Recall, the custom implementation allows us to precisely define how each sample contributes to the final aggregated score. For instance, in our `PrecisionMetric`, samples where the model predicts 'negative' are assigned a score of `0.0` with a reason indicating they are not applicable for precision calculation (as precision focuses on positive predictions). Opik handles the correct aggregation of these `0.0` values based on the metric's definition.

In essence, while the underlying concepts of these metrics are universal, creating custom implementations ensured seamless integration with Opik's unique per-sample evaluation and reporting mechanism.

